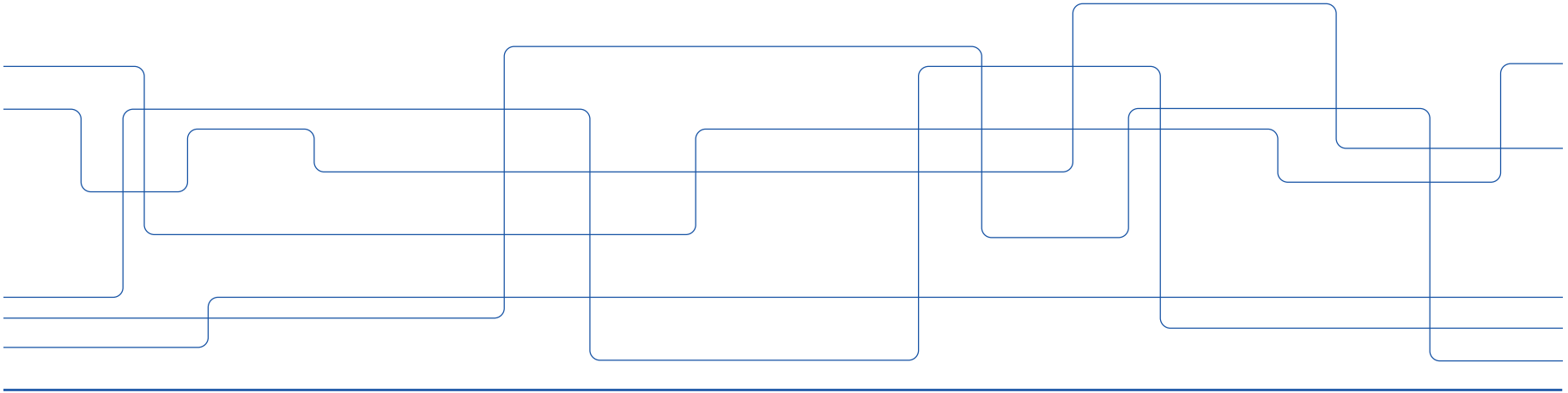
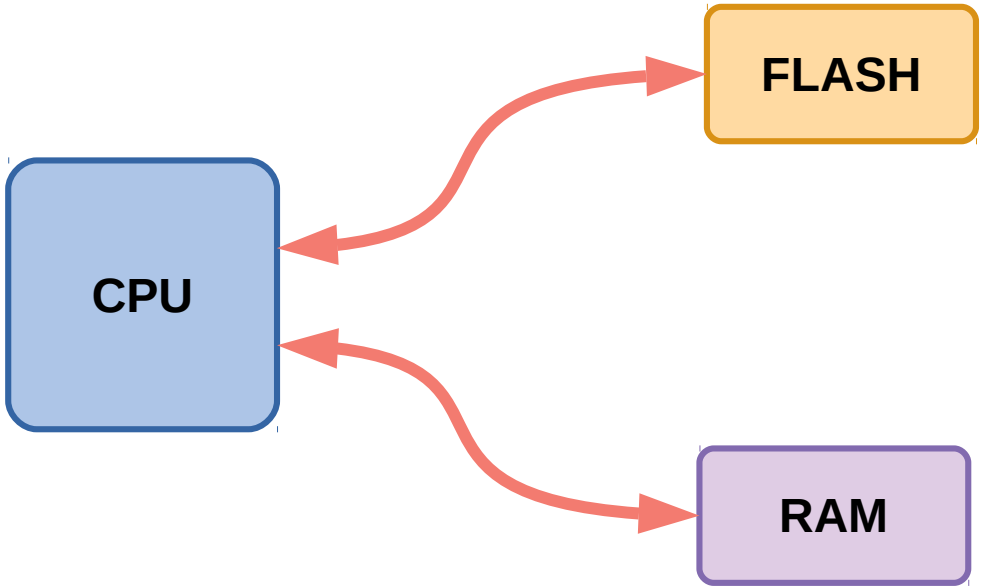
Direct Memory Access

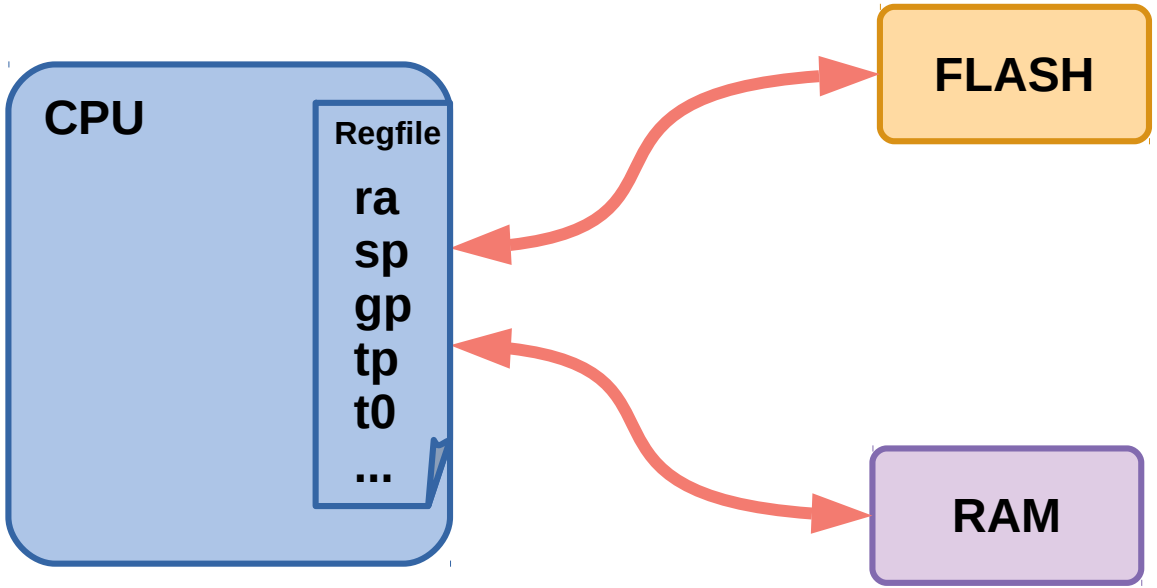
Introduction



Minneshantering i ett datorsystem

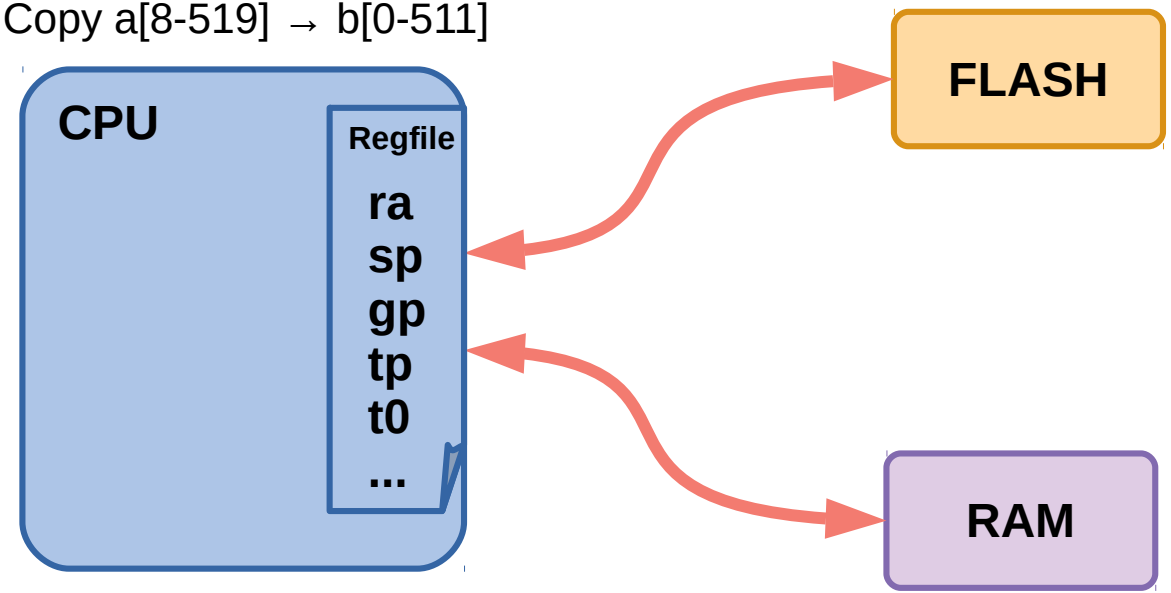


Minneshantering i ett datorsystem



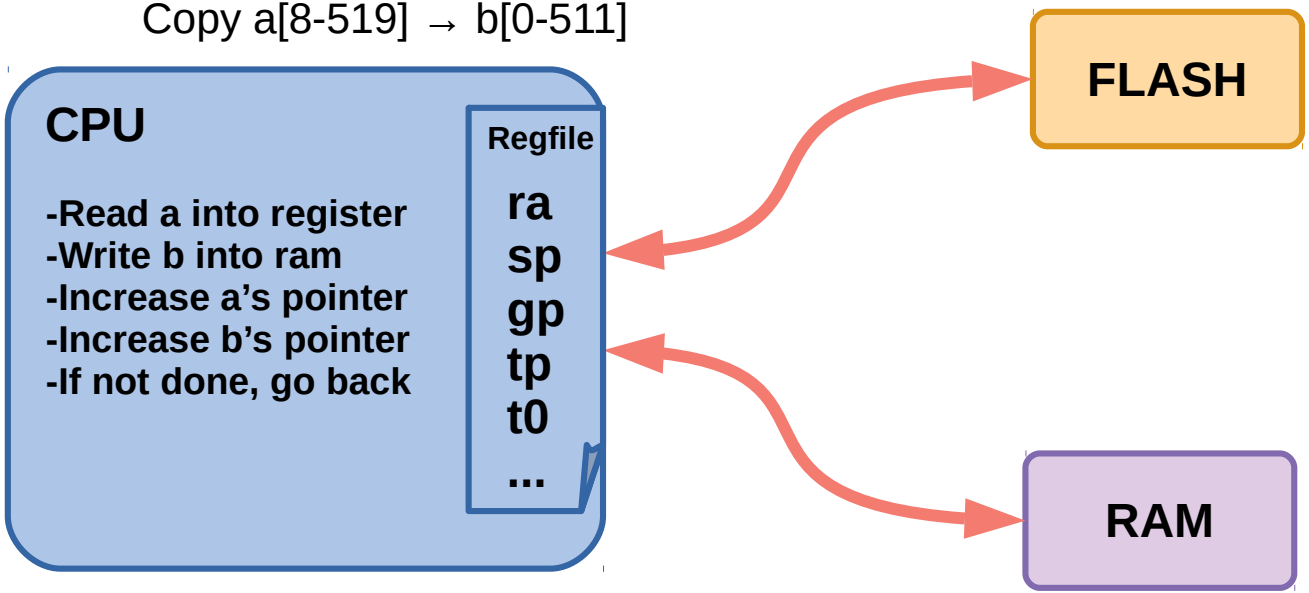
Minneshantering i ett datorsystem

Copy a[8-519] → b[0-511]



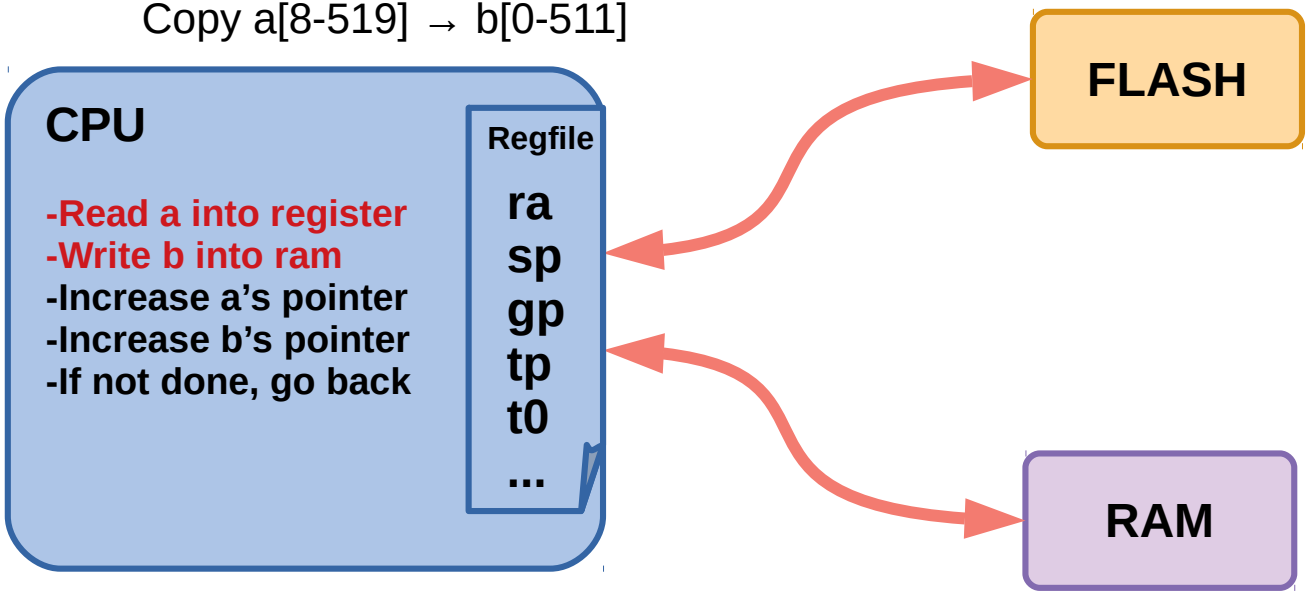
Minneshantering i ett datorsystem

Copy a[8-519] → b[0-511]

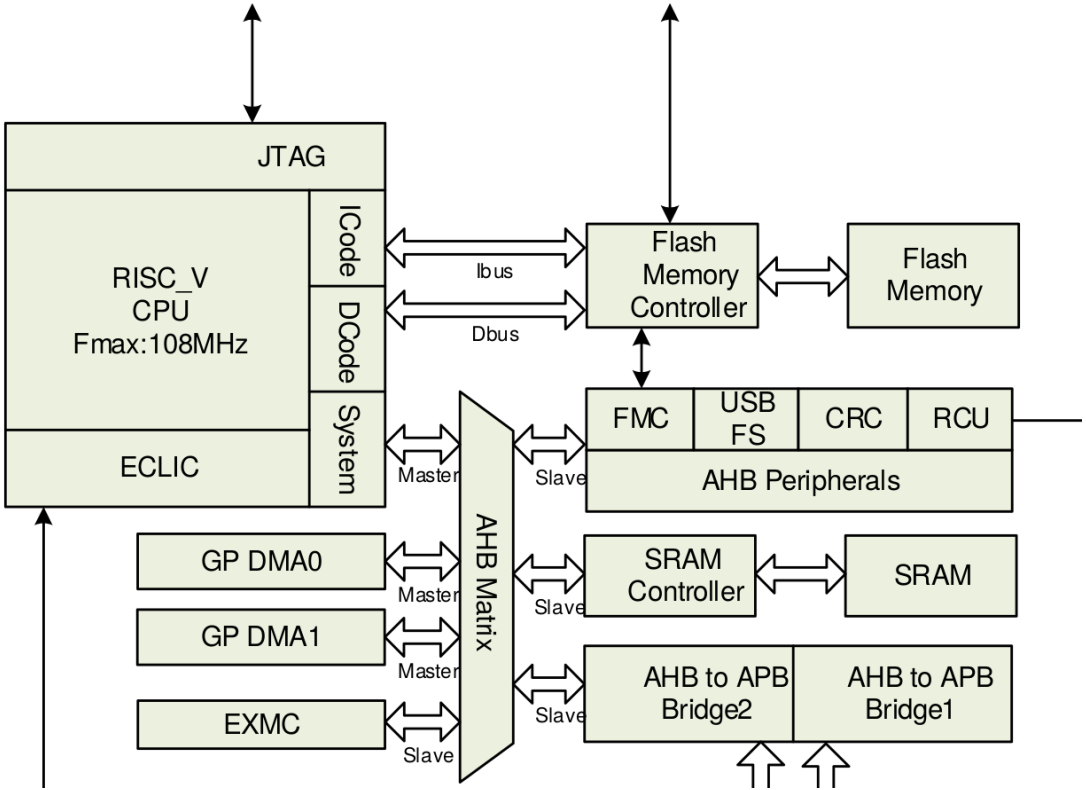


Minneshantering i ett datorsystem

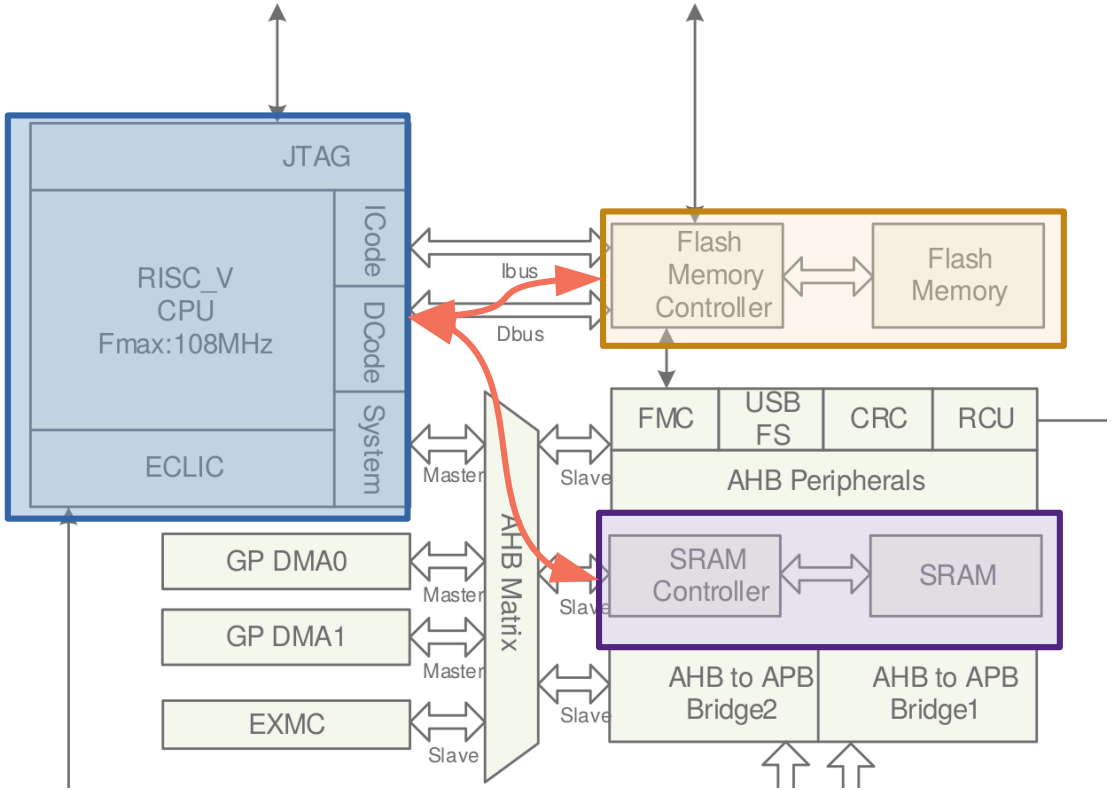
Copy a[8-519] → b[0-511]



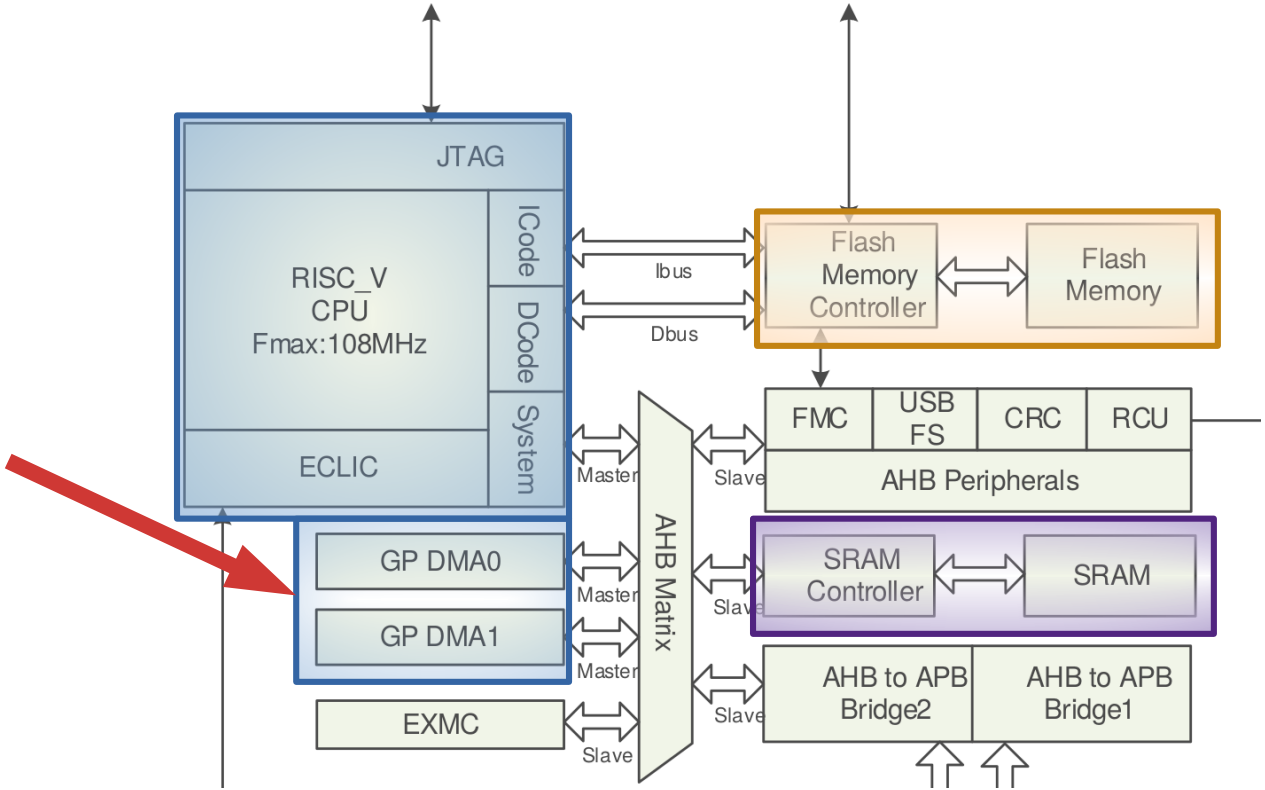
I GD32VF103


JTAG
RISC_V
CPU
Fmax:108MHz
ICode
DCode
System
ECLIC
Ibus
Dbus
Flash
Memory
Controller
Flash
Memory
FMC
USB
FS
CRC
RCU
AHB Peripherals
Master
Slave
AHB Matrix
GP DMA0
GP DMA1
EXMC
SRAM
Controller
SRAM
AHB to APB
Bridge2
AHB to APB
Bridge1

I GD32VF103


JTAG
RISC_V
CPU
Fmax:108MHz
ICode
DCode
System
ECLIC
Ibus
Dbus
Flash
Memory
Controller
Flash
Memory
FMC
USB
FS
CRC
RCU
AHB Peripherals
Master
Slave
AHB Matrix
GP DMA0
GP DMA1
EXMC
SRAM
Controller
SRAM
AHB to APB
Bridge2
AHB to APB
Bridge1

I GD32VF103


JTAG
RISC_V
CPU
Fmax:108MHz
ICode
DCode
System
ECLIC
Ibus
Dbus
Flash
Memory
Controller
Flash
Memory
FMC
USB
FS
CRC
RCU
AHB Peripherals
Master
Slave
AHB Matrix
GP DMA0
GP DMA1
EXMC
SRAM
Controller
SRAM
AHB to APB
Bridge2
AHB to APB
Bridge1

9. Direct memory access controller (DMA)

9.1. Overview

The direct memory access (DMA) controller provides a hardware method of transferring data between peripherals and/or memory without intervention from the CPU...

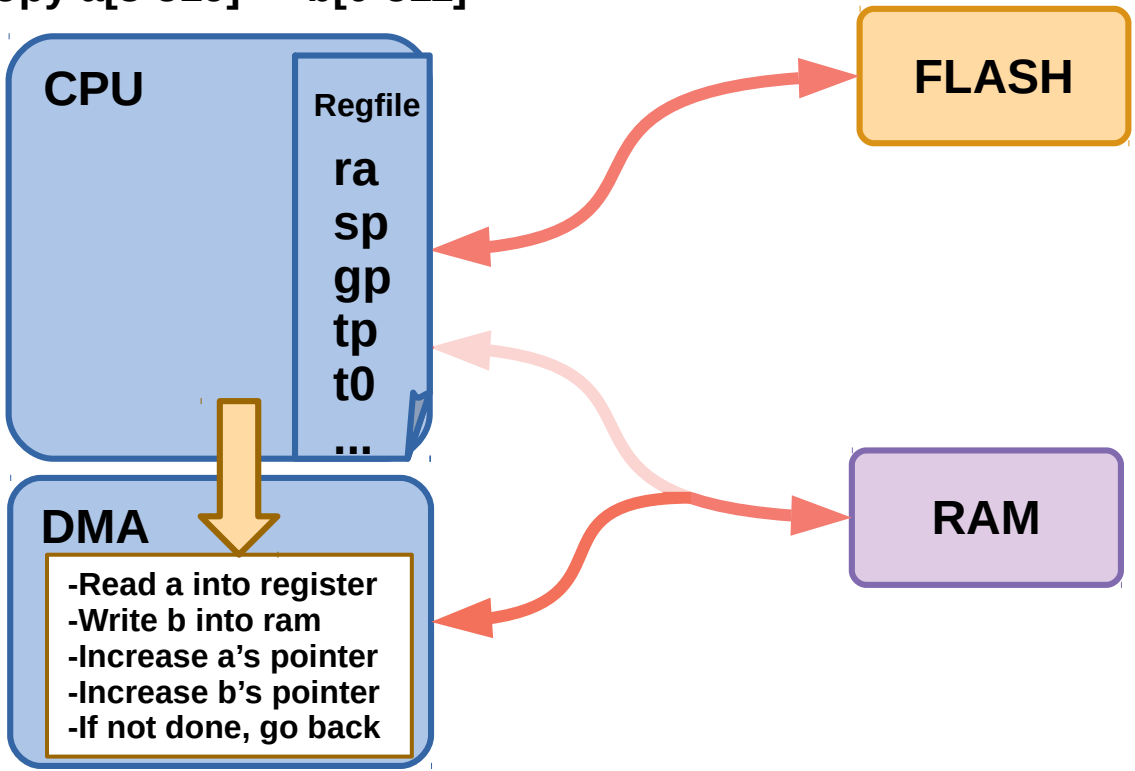
9. Direct memory access controller (DMA)

9.1. Overview

The direct memory access (DMA) controller provides a hardware method of transferring data between peripherals and/or memory **without intervention from the CPU...**

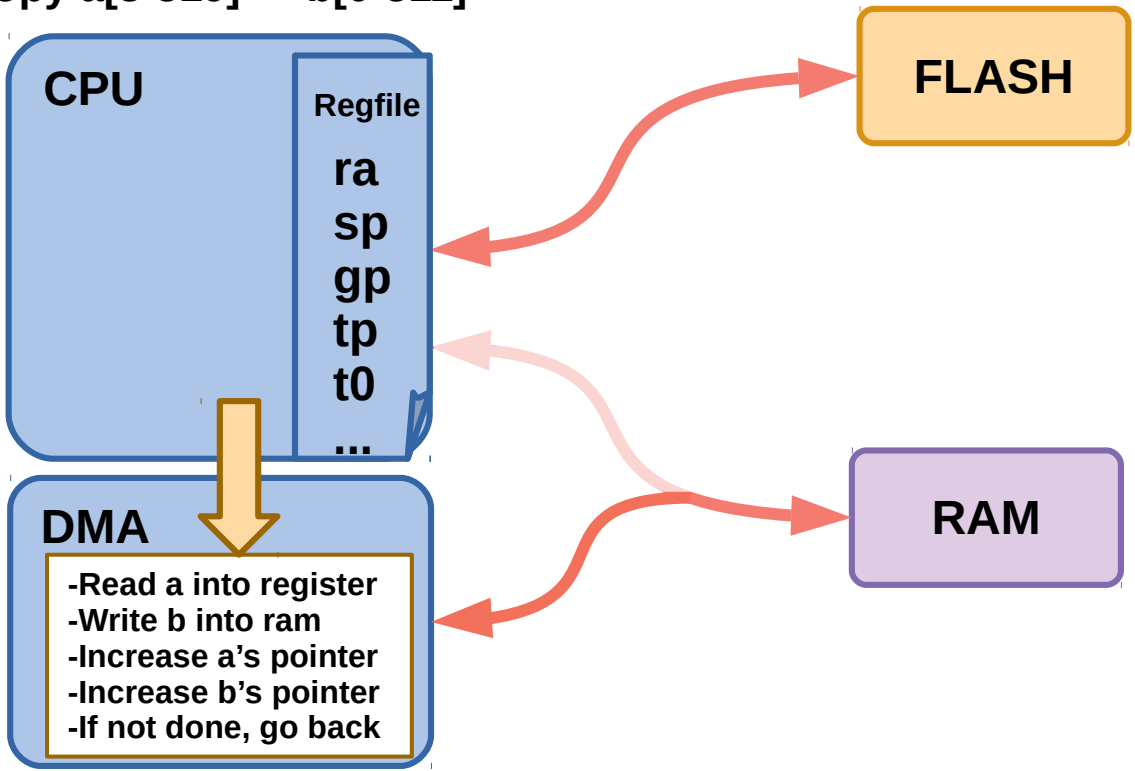
DMA

Copy a[8-519] → b[0-511]



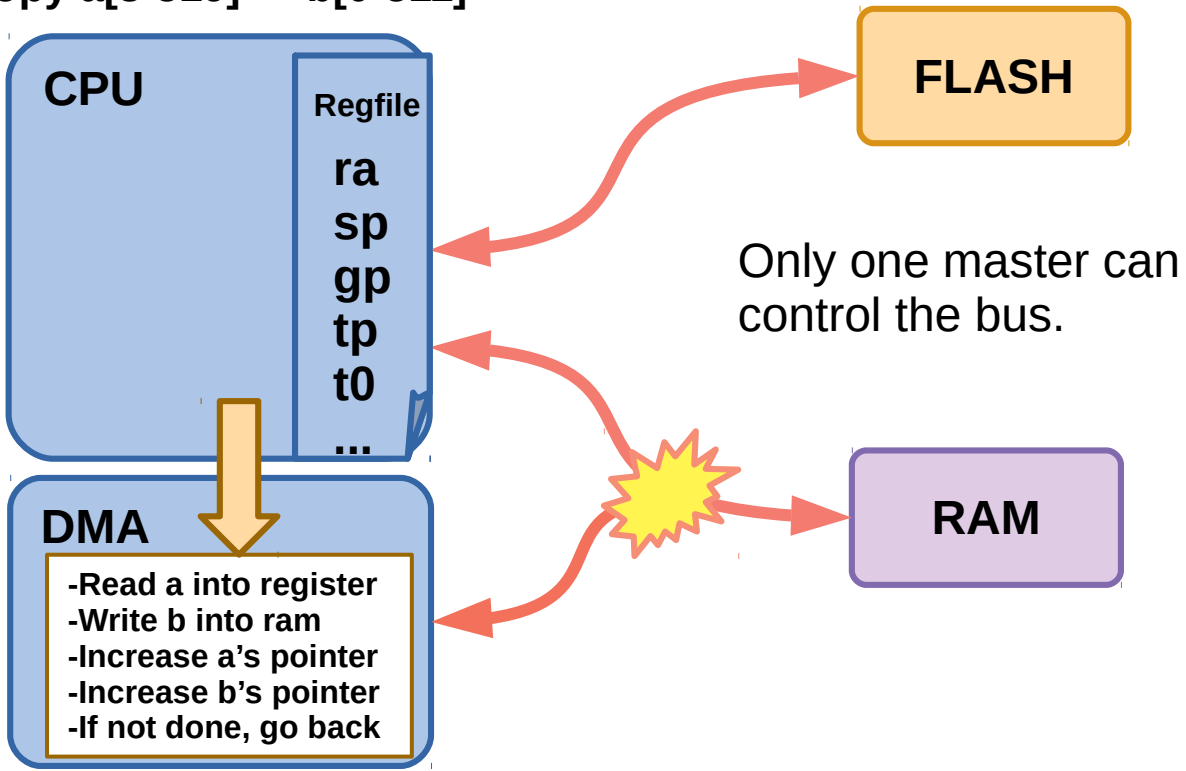
DMA

Copy a[8-519] → b[0-511]



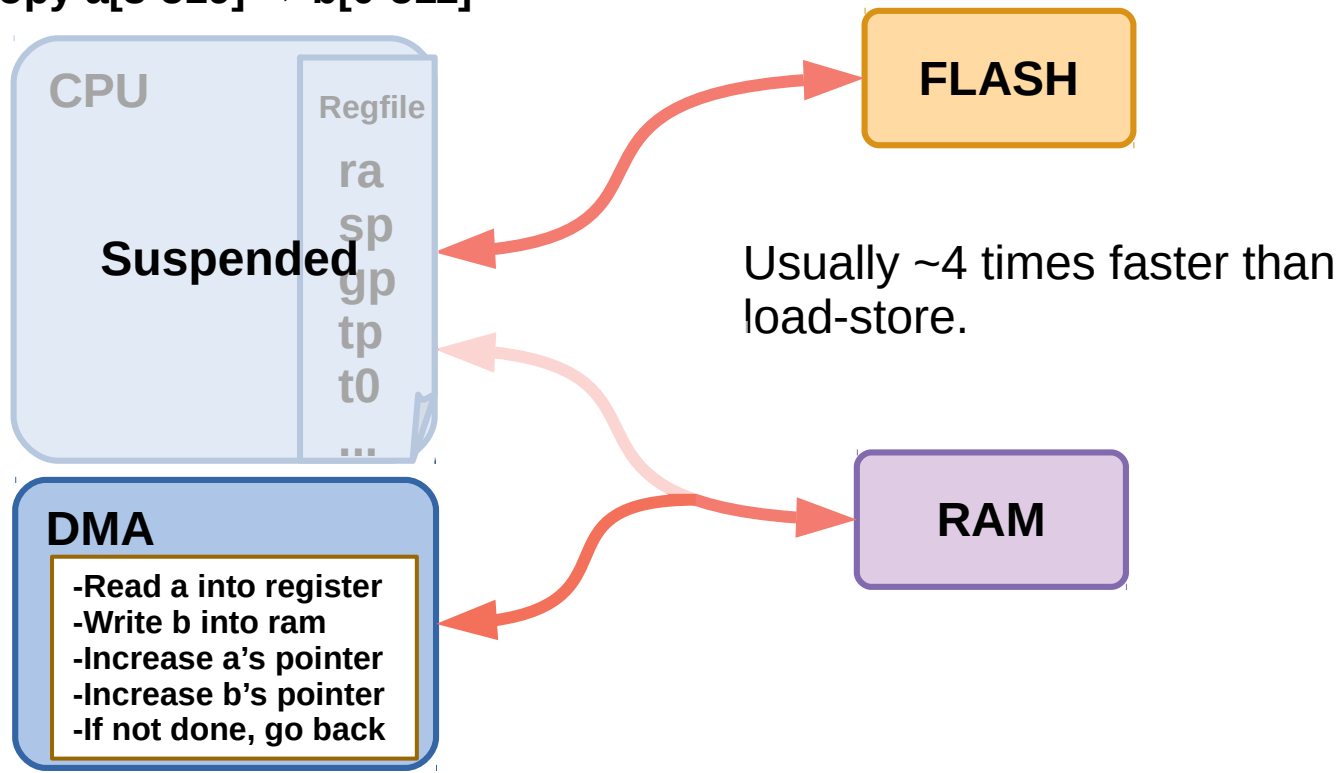
DMA

Copy a[8-519] → b[0-511]



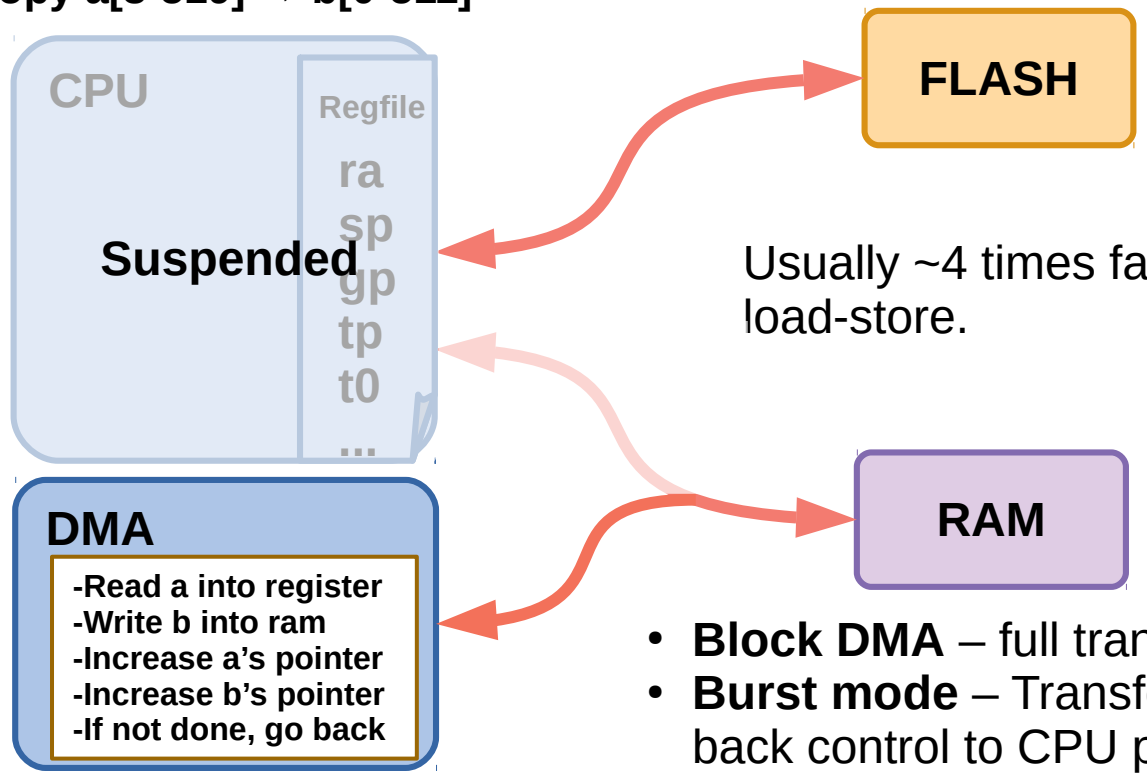
Bus control

Copy a[8-519] → b[0-511]



Bus control

Copy a[8-519] → b[0-511]

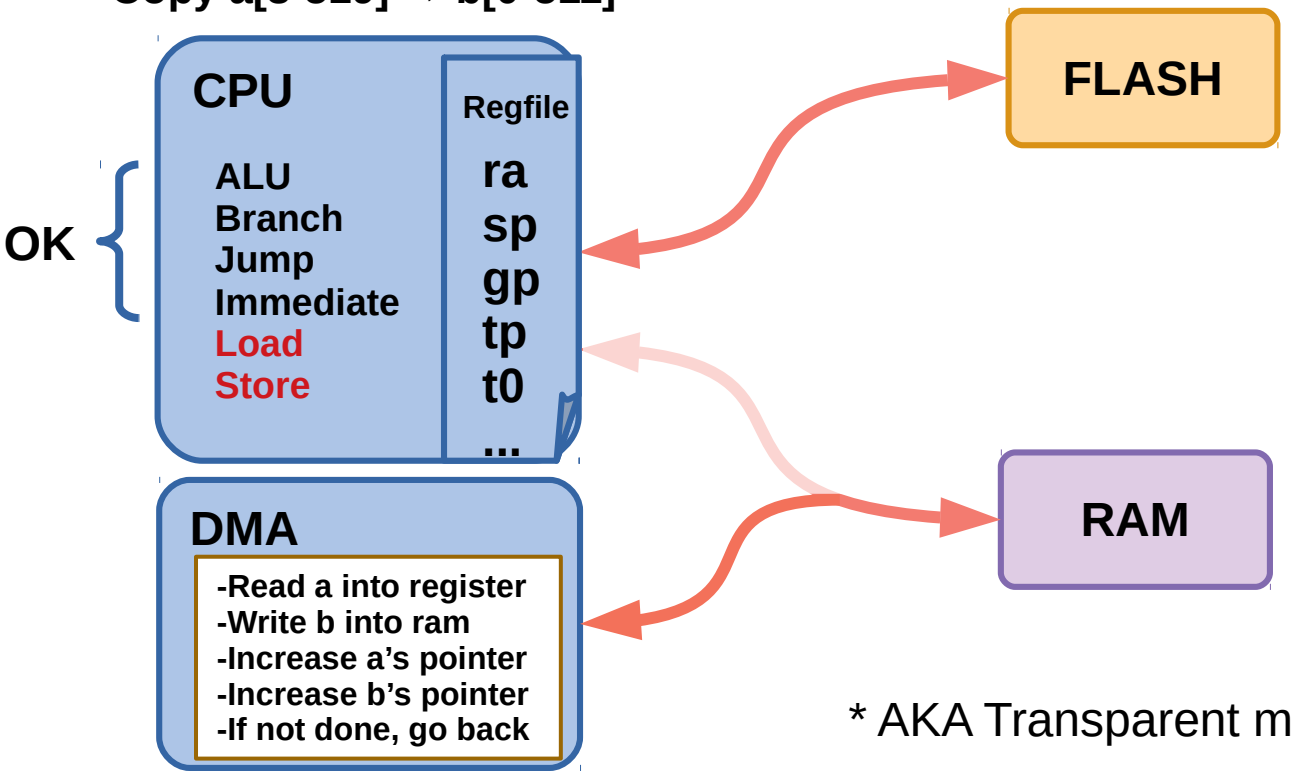


Usually ~4 times faster than load-store.

- **Block DMA** – full transfer in one go
- **Burst mode** – Transfer a bit then give back control to CPU periodically

Bus control – cycle stealing

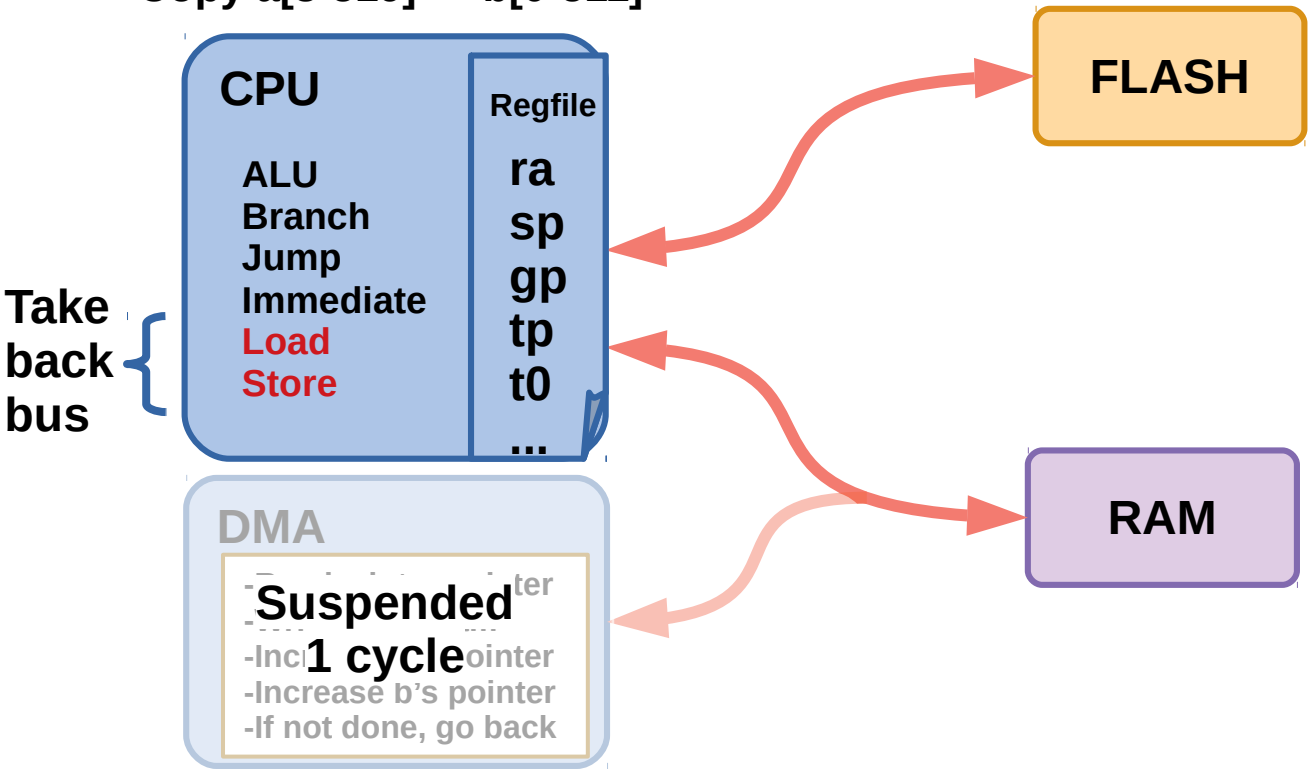
Copy a[8-519] → b[0-511]



* AKA Transparent mode

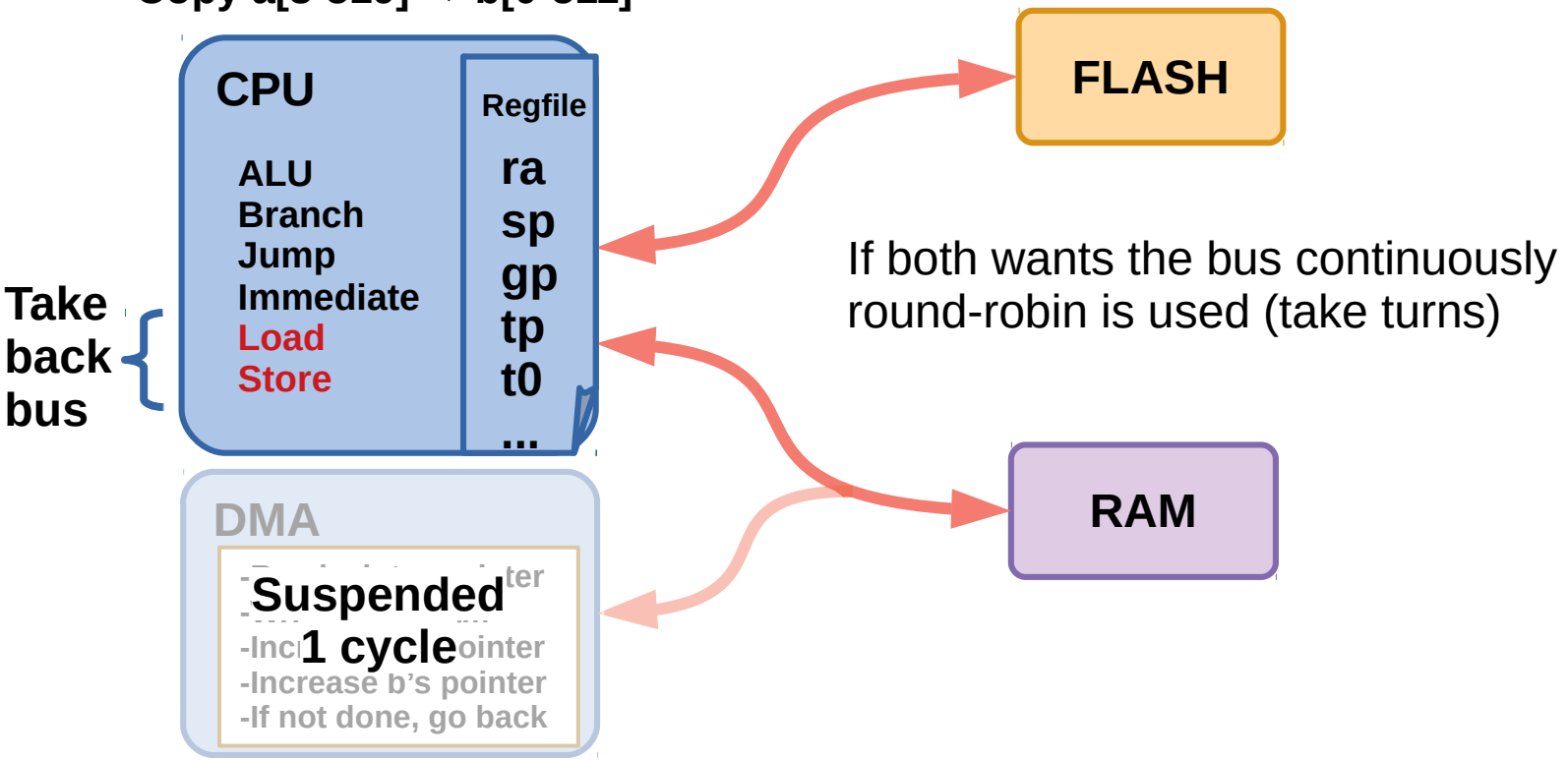
Bus control – cycle stealing

Copy a[8-519] → b[0-511]

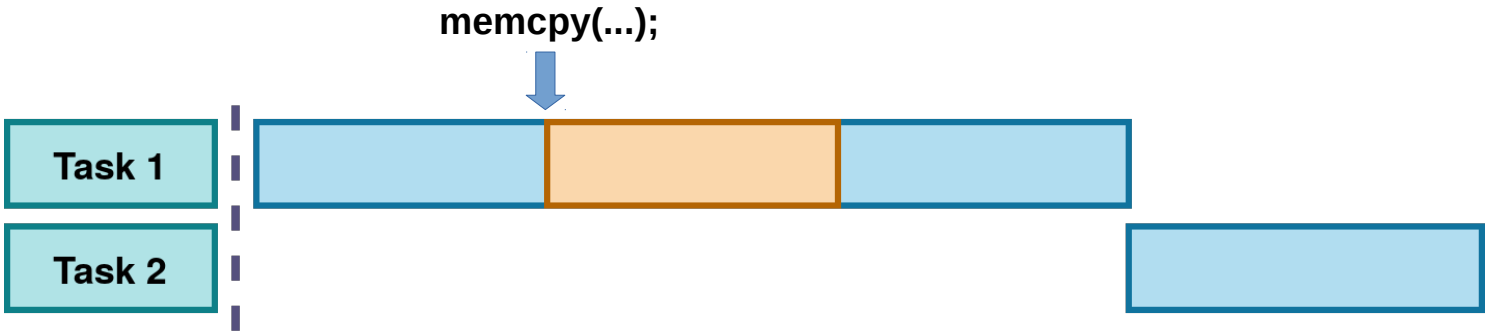


Bus control – cycle stealing

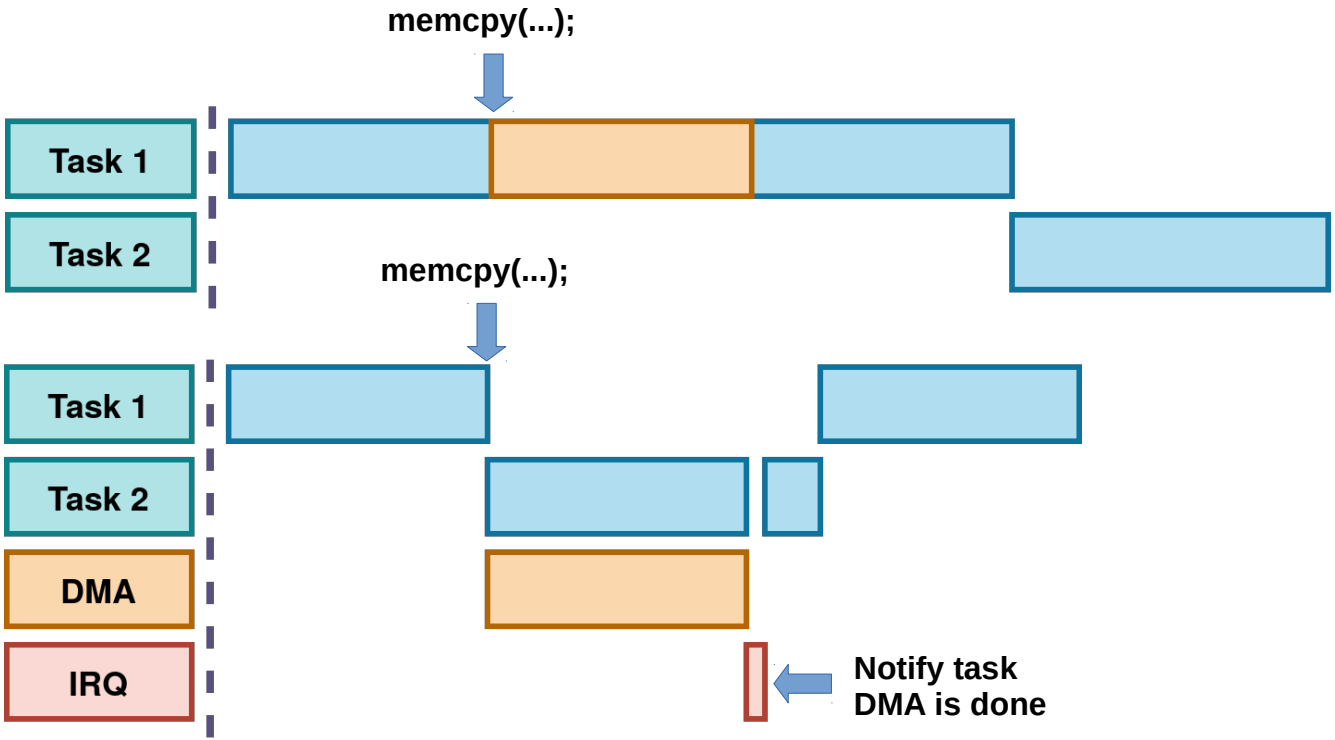
Copy a[8-519] → b[0-511]



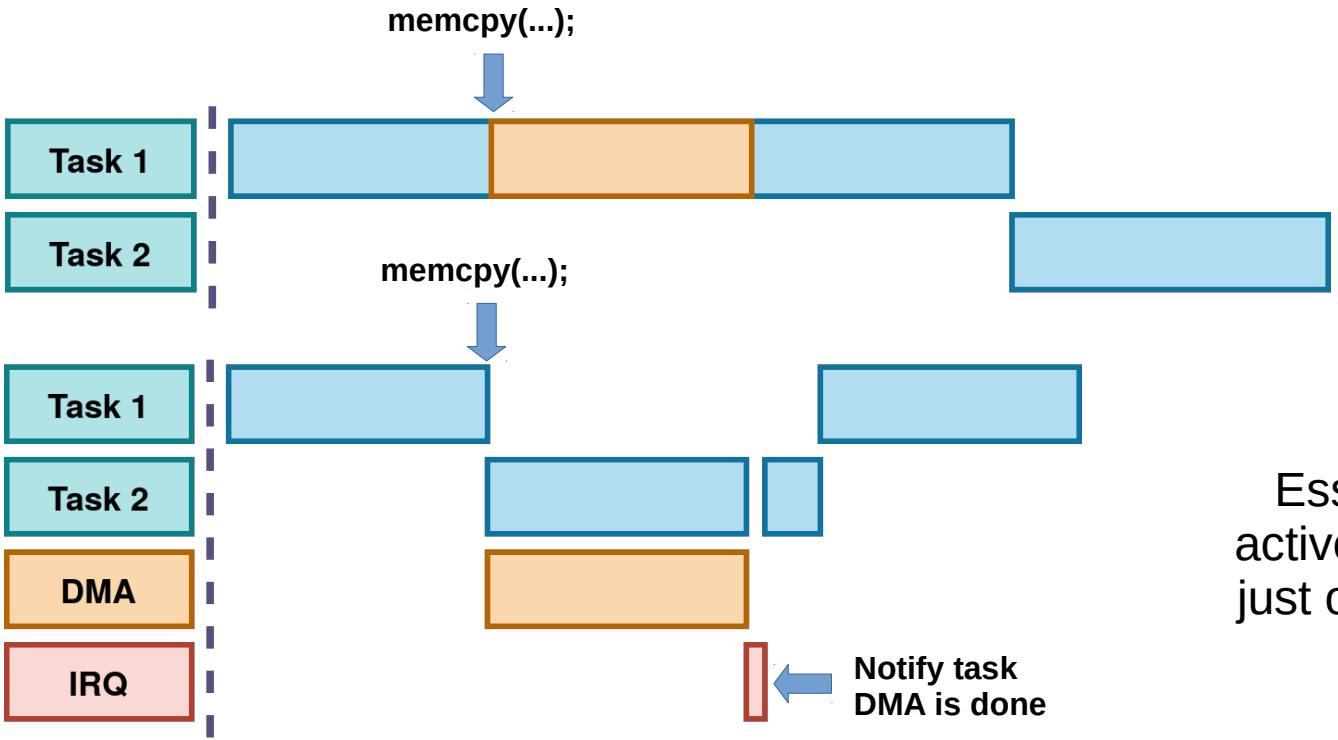
In a multitasking environment



In a multitasking environment

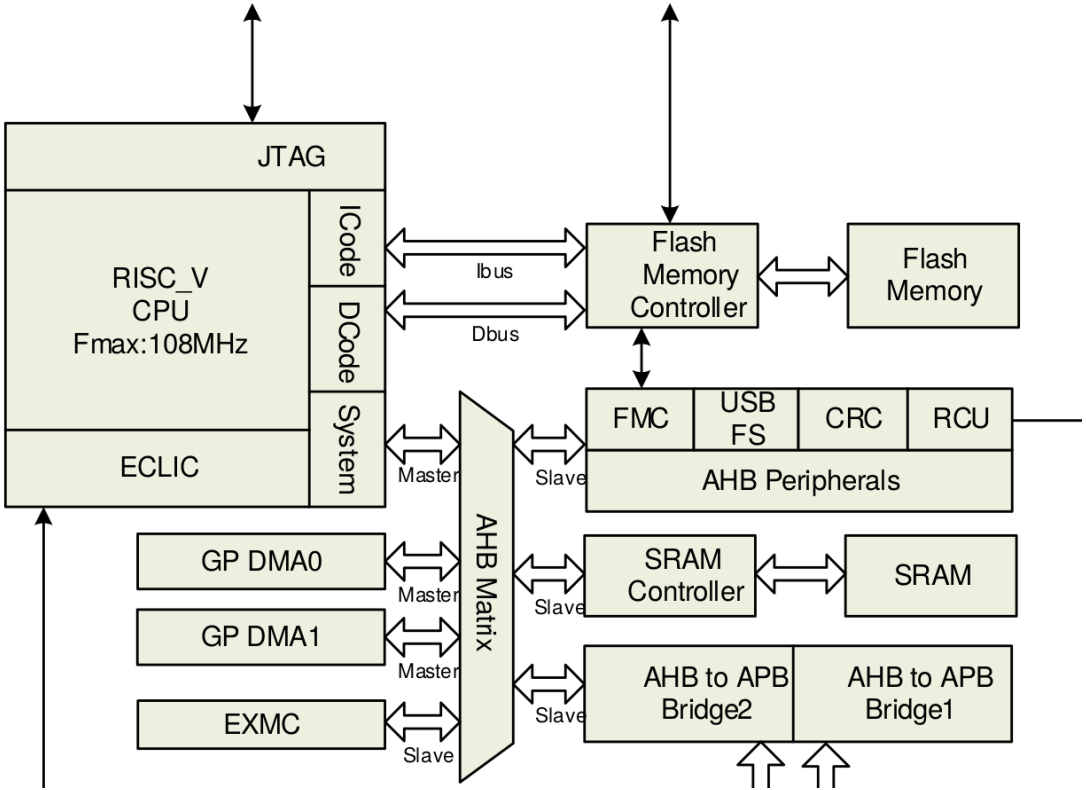


In a multitasking environment

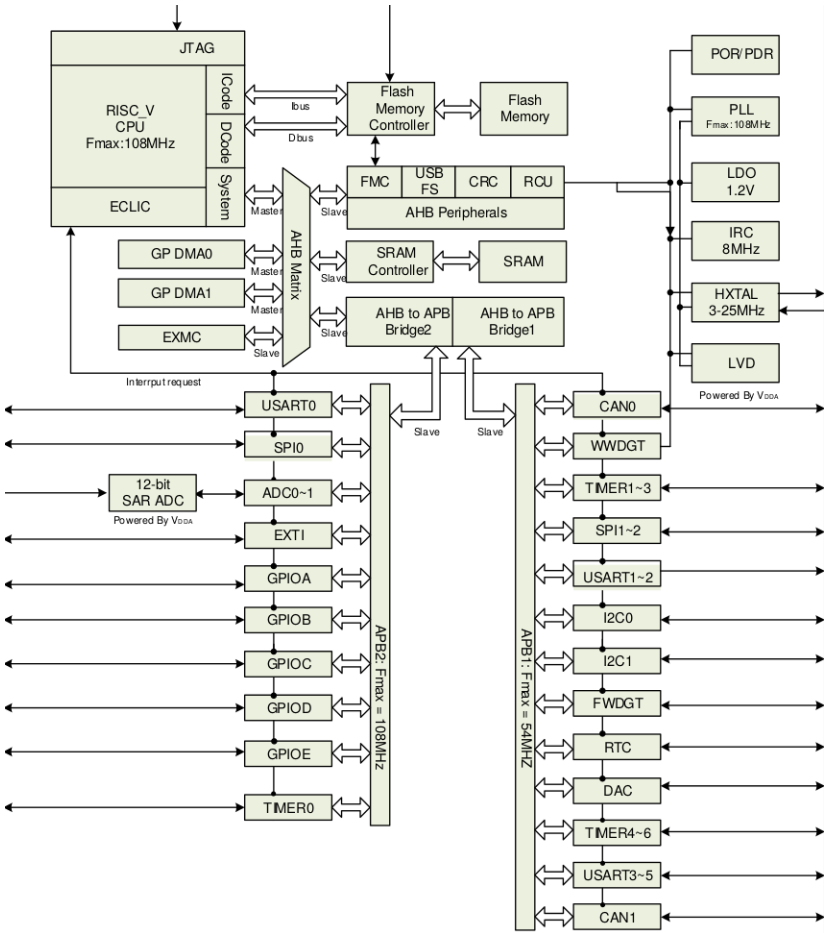


Essentially two active threads with just one processor core!

I GD32VF103

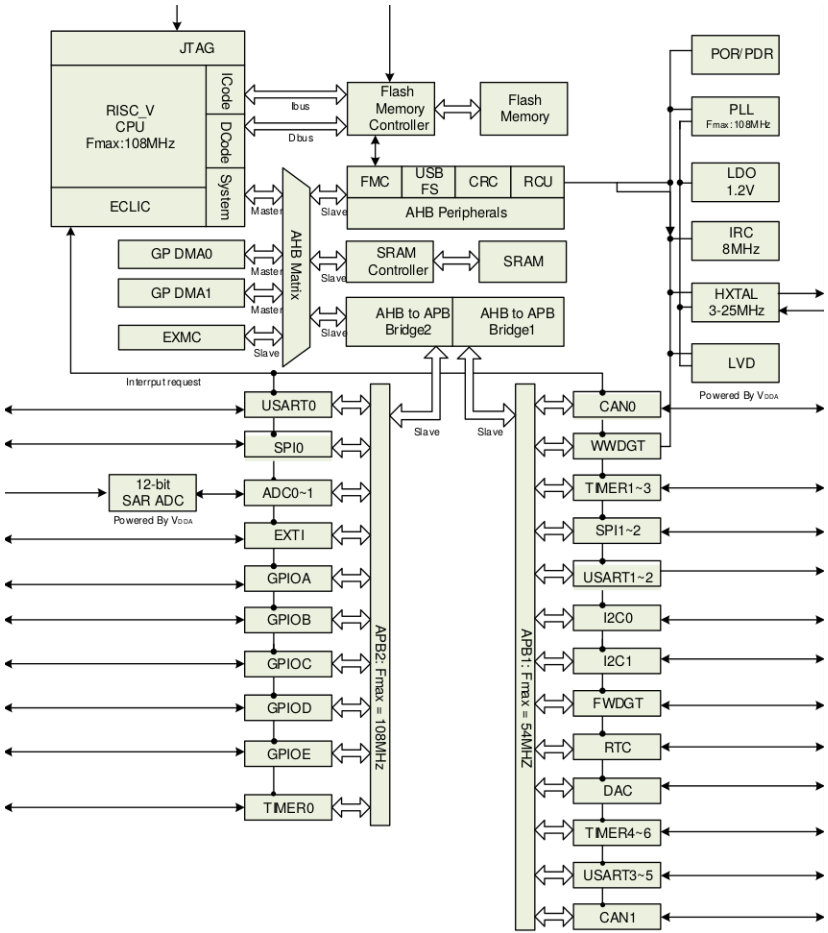

JTAG
RISC_V
CPU
Fmax:108MHz
ICode
DCode
System
ECLIC
Ibus
Dbus
Flash
Memory
Controller
Flash
Memory
FMC
USB
FS
CRC
RCU
AHB Peripherals
Master
Slave
AHB Matrix
GP DMA0
GP DMA1
EXMC
SRAM
Controller
SRAM
AHB to APB
Bridge2
AHB to APB
Bridge1

I GD32VF103

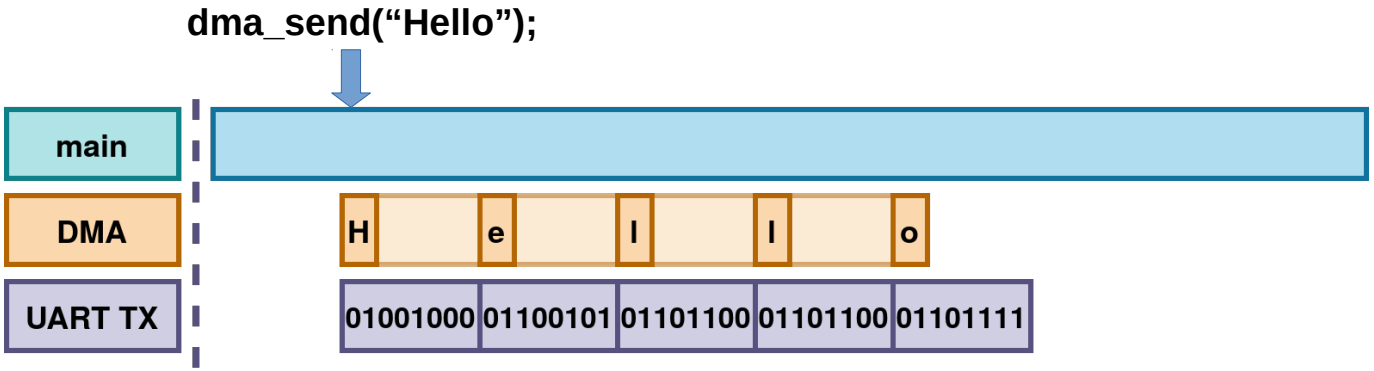

JTAG
RISC_V CPU Fmax:108MHz
ICode
DCode
System
ECLIC
Ibus
Dbus
Flash Memory Controller
Flash Memory
FMC
USB FS
CRC
RCU
AHB Peripherals
Master
Slave
AHB Matrix
GP DMA0
GP DMA1
EXMC
SRAM Controller
SRAM
AHB to APB Bridge2
AHB to APB Bridge1
Interrput request
POR/PDR
PLL
Fmax:108MHz
LDO 1.2V
IRC 8MHz
HXTAL 3-25MHz
LVD
Powered By VDDA
USART0
SPI0
12-bit SAR ADC
ADC0~1
EXTI
GPIOA
GPIOB
GPIOC
GPIOD
GPIOE
TIMER0
APB2: Fmax = 108MHz
CAN0
WWDGT
TIMER1~3
SPI1~2
USART1~2
I2C0
I2C1
FWDGT
RTC
DAC
TIMER4~6
USART3~5
CAN1
APB1: Fmax = 54MHz

I GD32VF103

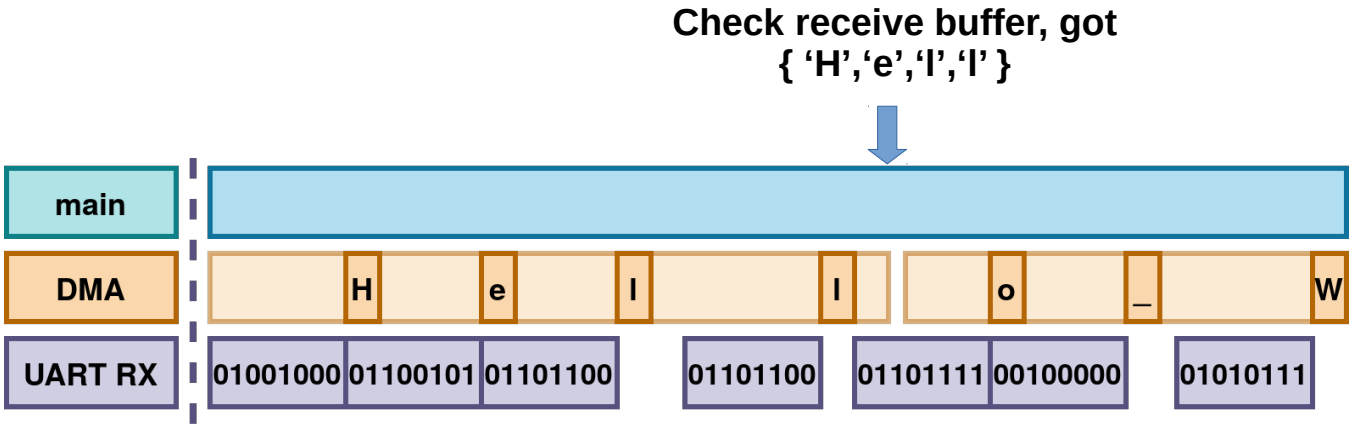
...provides a hardware method of transferring data between **peripherals** and/or memory...



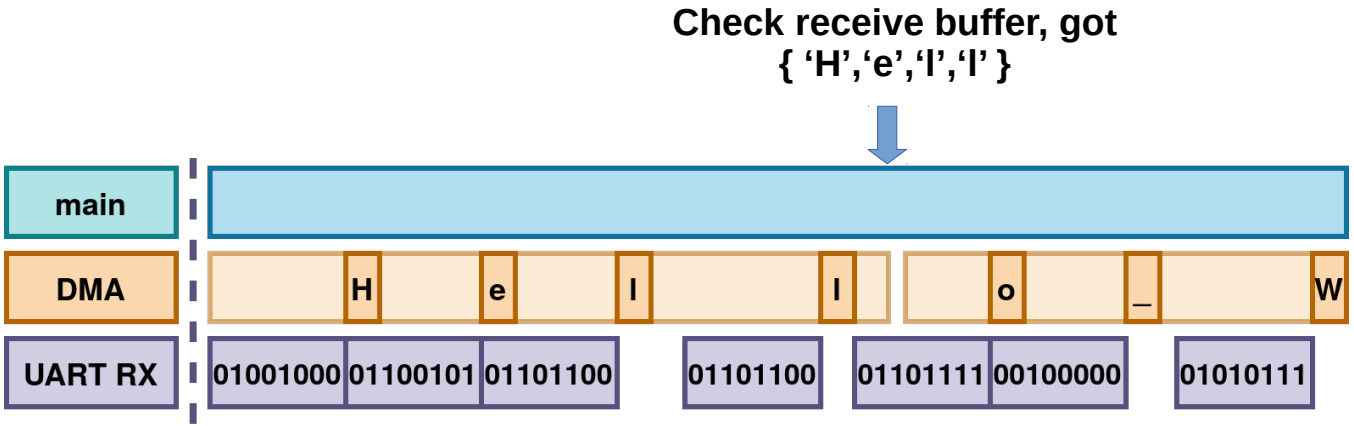
UART – Transmit buffer



UART – Receive buffer



UART – Receive buffer



Guarantees that we don’t miss any characters, unlike when polling!

UART – Example code

First initialize UART

```
/* USART configure */
usart_deinit(USART0);
usart_baudrate_set(USART0, 115200U);
usart_word_length_set(USART0, USART_WL_8BIT);
usart_stop_bit_set(USART0, USART_STB_1BIT);
usart_parity_config(USART0, USART_PM_NONE);
usart_hardware_flow_rts_config(USART0, USART_RTS_DISABLE);
usart_hardware_flow_cts_config(USART0, USART_CTS_DISABLE);
usart_receive_config(USART0, USART_RECEIVE_ENABLE);
usart_transmit_config(USART0, USART_TRANSMIT_ENABLE);
usart_enable(USART0);
```

Enable DMA

```
usart_dma_transmit_config(USART0, USART_DENT_ENABLE);
usart_dma_receive_config(USART0, USART_DENR_ENABLE);
```

UART – Example code RX

RX to RAM

```
dma_deinit(DMA0, DMA_CH4);
dma_init_struct.direction = DMA_PERIPHERAL_TO_MEMORY;
dma_init_struct.memory_addr = (uint32_t)rx_dma_buffer;
dma_init_struct.memory_inc = DMA_MEMORY_INCREASE_ENABLE;
dma_init_struct.memory_width = DMA_MEMORY_WIDTH_8BIT;
dma_init_struct.number = LIO_BT_BUFFER_SIZE;
dma_init_struct.periph_addr = (uint32_t)&USART_DATA(USART0);
dma_init_struct.periph_inc = DMA_PERIPH_INCREASE_DISABLE;
dma_init_struct.periph_width = DMA_PERIPHERAL_WIDTH_8BIT;
dma_init_struct.priority = DMA_PRIORITY_ULTRA_HIGH;
dma_init(DMA0, DMA_CH4, &dma_init_struct);
    /* configure DMA mode */
dma_circulation_disable(DMA0, DMA_CH4);
    /* enable DMA channel4 */
dma_channel_enable(DMA0, DMA_CH4);
```

UART – Example code RX

Destination pointer,
Initialized with ‘\0’

```
dma_deinit(DMA0, DMA_CH4);
dma_init_struct.direction = DMA_PERIPHERAL_TO_MEMORY;
dma_init_struct.memory_addr = (uint32_t)rx_dma_buffer;
dma_init_struct.memory_inc = DMA_MEMORY_INCREASE_ENABLE;
dma_init_struct.memory_width = DMA_MEMORY_WIDTH_8BIT;
dma_init_struct.number = LIO_BT_BUFFER_SIZE;
dma_init_struct.periph_addr = (uint32_t)&USART_DATA(USART0);
dma_init_struct.periph_inc = DMA_PERIPH_INCREASE_DISABLE;
dma_init_struct.periph_width = DMA_PERIPHERAL_WIDTH_8BIT;
dma_init_struct.priority = DMA_PRIORITY_ULTRA_HIGH;
dma_init(DMA0, DMA_CH4, &dma_init_struct);
    /* configure DMA mode */
dma_circulation_disable(DMA0, DMA_CH4);
    /* enable DMA channel4 */
dma_channel_enable(DMA0, DMA_CH4);
```

UART – Example code RX

```
dma_deinit(DMA0, DMA_CH4);
dma_init_struct.direction = DMA_PERIPHERAL_TO_MEMORY;
dma_init_struct.memory_addr = (uint32_t)rx_dma_buffer;
dma_init_struct.memory_inc = DMA_MEMORY_INCREASE_ENABLE;
dma_init_struct.memory_width = DMA_MEMORY_WIDTH_8BIT;
dma_init_struct.number = LIO_BT_BUFFER_SIZE;
dma_init_struct.periph_addr = (uint32_t)&USART_DATA(USART0);
dma_init_struct.periph_inc = DMA_PERIPH_INCREASE_DISABLE;
dma_init_struct.periph_width = DMA_PERIPHERAL_WIDTH_8BIT;
dma_init_struct.priority = DMA_PRIORITY_ULTRA_HIGH;
dma_init(DMA0, DMA_CH4, &dma_init_struct);
    /* configure DMA mode */
dma_circulation_disable(DMA0, DMA_CH4);
    /* enable DMA channel4 */
dma_channel_enable(DMA0, DMA_CH4);
```

Peripheral address

UART – Example code RX

```
dma_deinit(DMA0, DMA_CH4);
dma_init_struct.direction = DMA_PERIPHERAL_TO_MEMORY;
dma_init_struct.memory_addr = (uint32_t)rx_dma_buffer;
dma_init_struct.memory_inc = DMA_MEMORY_INCREASE_ENABLE;
dma_init_struct.memory_width = DMA_MEMORY_WIDTH_8BIT;
dma_init_struct.number = LIO_BT_BUFFER_SIZE;
dma_init_struct.periph_addr = (uint32_t)&USART_DATA(USART0);
dma_init_struct.periph_inc = DMA_PERIPH_INCREASE_DISABLE;
dma_init_struct.periph_width = DMA_PERIPHERAL_WIDTH_8BIT;
dma_init_struct.priority = DMA_PRIORITY_ULTRA_HIGH;
dma_init(DMA0, DMA_CH4, &dma_init_struct);
    /* configure DMA mode */
dma_circulation_disable(DMA0, DMA_CH4);
    /* enable DMA channel4 */
dma_channel_enable(DMA0, DMA_CH4);
```

UART – Example code RX

```
if(rx_dma_buffer[0] != '\0'){
    int i = 0;
    for(; i < LIO_BT_BUFFER_SIZE && rx_dma_buffer[i] != 0 && i < size - 1; i++){
        buffer[i] = rx_dma_buffer[i];
        rx_dma_buffer[i] = '\0';
    }
    buffer[i] = '\0';
}
```

UART – Example code RX

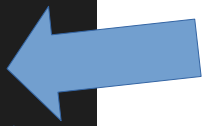
```
if(rx_dma_buffer[0] != '\0'){
    int i = 0;
    for(; i < LIO_BT_BUFFER_SIZE && rx_dma_buffer[i] != 0 && i < size - 1; i++){
        buffer[i] = rx_dma_buffer[i];
        rx_dma_buffer[i] = '\0';
    }
    buffer[i] = '\0';
}
```

Then initialize DMA again

UART – Example code

```
size_t lio_send_bt(char* message, uint32_t size)
{
    for(int i = 0; i < size && i < LIO_BT_BUFFER_SIZE; i++) {
        tx_dma_buffer[i] = message[i];
    }

    dma_parameter_struct dma_init_struct;
    dma_deinit(DMA0, DMA_CH3);
    dma_init_struct.direction = DMA_MEMORY_TO_PERIPHERAL;
    dma_init_struct.memory_addr = (uint32_t)tx_dma_buffer;
    dma_init_struct.memory_inc = DMA_MEMORY_INCREASE_ENABLE;
    dma_init_struct.memory_width = DMA_MEMORY_WIDTH_8BIT;
    dma_init_struct.number = 1;
    dma_init_struct.periph_addr = (uint32_t)&USART_DATA(USART0);
    dma_init_struct.periph_inc = DMA_PERIPH_INCREASE_DISABLE;
    dma_init_struct.periph_width = DMA_PERIPHERAL_WIDTH_8BIT;
    dma_init_struct.priority = DMA_PRIORITY_ULTRA_HIGH;
    dma_init(DMA0, DMA_CH3, &dma_init_struct);
    /* configure DMA mode */
    dma_circulation_disable(DMA0, DMA_CH3);
    /* enable DMA channel3 */
    dma_channel_enable(DMA0, DMA_CH3);
}
```



Copy over to buffer

UART – Example code

```
size_t lio_send_bt(char* message, uint32_t size)
{
    for(int i = 0; i < size && i < LIO_BT_BUFFER_SIZE; i++) {
        tx_dma_buffer[i] = message[i];
    }

    dma_parameter_struct dma_init_struct;
    dma_deinit(DMA0, DMA_CH3);
    dma_init_struct.direction = DMA_MEMORY_TO_PERIPHERAL;
    dma_init_struct.memory_addr = (uint32_t)tx_dma_buffer;
    dma_init_struct.memory_inc = DMA_MEMORY_INCREASE_ENABLE;
    dma_init_struct.memory_width = DMA_MEMORY_WIDTH_8BIT;
    dma_init_struct.number = 1;
    dma_init_struct.periph_addr = (uint32_t)&USART_DATA(USART0);
    dma_init_struct.periph_inc = DMA_PERIPH_INCREASE_DISABLE;
    dma_init_struct.periph_width = DMA_PERIPHERAL_WIDTH_8BIT;
    dma_init_struct.priority = DMA_PRIORITY_ULTRA_HIGH;
    dma_init(DMA0, DMA_CH3, &dma_init_struct);
    /* configure DMA mode */
    dma_circulation_disable(DMA0, DMA_CH3);
    /* enable DMA channel3 */
    dma_channel_enable(DMA0, DMA_CH3);
}
```

Initialize

UART – Example code

```
size_t lio_send_bt(char* message, uint32_t size)
{
    for(int i = 0; i < size && i < LIO_BT_BUFFER_SIZE; i++) {
        tx_dma_buffer[i] = message[i];
    }

    dma_parameter_struct dma_init_struct;
    dma_deinit(DMA0, DMA_CH3);
    dma_init_struct.direction = DMA_MEMORY_TO_PERIPHERAL;
    dma_init_struct.memory_addr = (uint32_t)tx_dma_buffer;
    dma_init_struct.memory_inc = DMA_MEMORY_INCREASE_ENABLE;
    dma_init_struct.memory_width = DMA_MEMORY_WIDTH_8BIT;
    dma_init_struct.number = 1;
    dma_init_struct.periph_addr = (uint32_t)&USART_DATA(USART0);
    dma_init_struct.periph_inc = DMA_PERIPH_INCREASE_DISABLE;
    dma_init_struct.periph_width = DMA_PERIPHERAL_WIDTH_8BIT;
    dma_init_struct.priority = DMA_PRIORITY_ULTRA_HIGH;
    dma_init(DMA0, DMA_CH3, &dma_init_struct);
    /* configure DMA mode */
    dma_circulation_disable(DMA0, DMA_CH3);
    /* enable DMA channel3 */
    dma_channel_enable(DMA0, DMA_CH3);
}
```

Enable

UART – Example code

```
size_t lio_send_bt(char* message, uint32_t size)
{
    for(int i = 0; i < size && i < LIO_BT_BUFFER_SIZE; i++) {
        tx_dma_buffer[i] = message[i];
    }

    dma_parameter_struct dma_init_struct;
    dma_deinit(DMA0, DMA_CH3);
    dma_init_struct.direction = DMA_MEMORY_TO_PERIPHERAL;
    dma_init_struct.memory_addr = (uint32_t)tx_dma_buffer;
    dma_init_struct.memory_inc = DMA_MEMORY_INCREASE_ENABLE;
    dma_init_struct.memory_width = DMA_MEMORY_WIDTH_8BIT;
    dma_init_struct.number = 1;
    dma_init_struct.periph_addr = (uint32_t)&USART_DATA(USART0);
    dma_init_struct.periph_inc = DMA_PERIPH_INCREASE_DISABLE;
    dma_init_struct.periph_width = DMA_PERIPHERAL_WIDTH_8BIT;
    dma_init_struct.priority = DMA_PRIORITY_ULTRA_HIGH;
    dma_init(DMA0, DMA_CH3, &dma_init_struct);
    /* configure DMA mode */
    dma_circulation_disable(DMA0, DMA_CH3);
    /* enable DMA channel3 */
    dma_channel_enable(DMA0, DMA_CH3);
}
```

Enable

UART – Example code

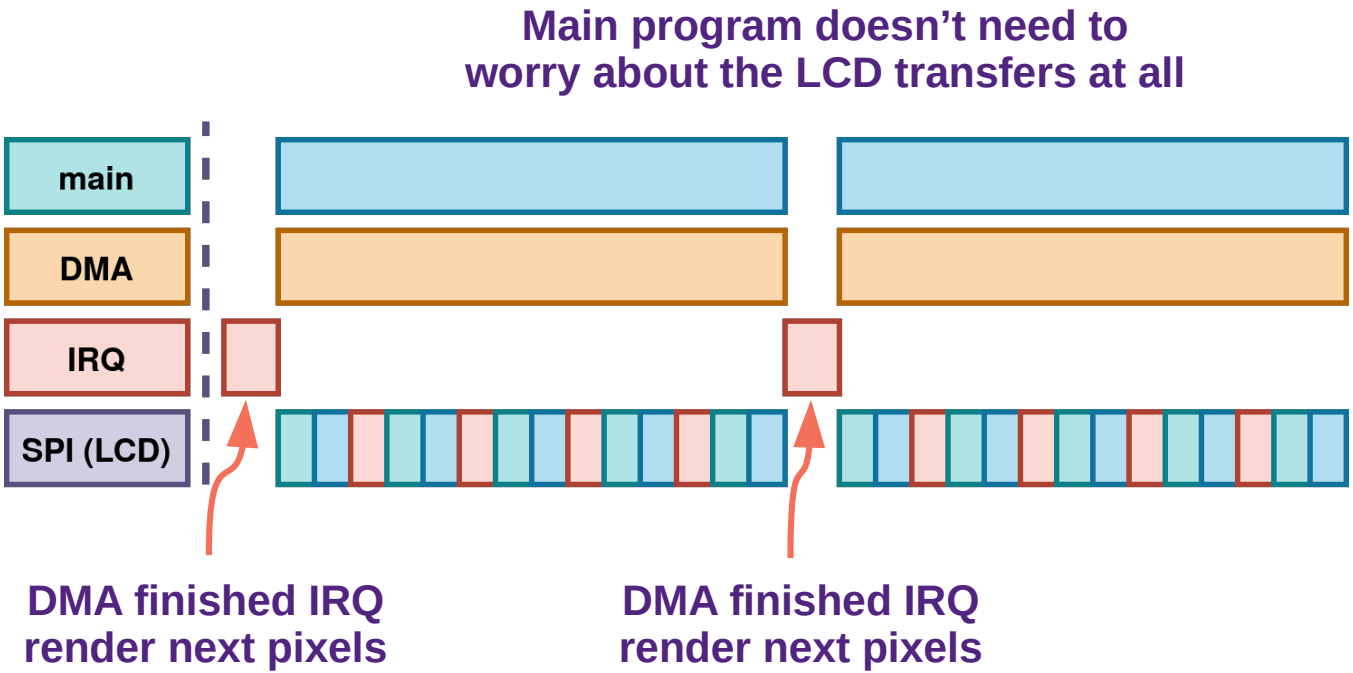
```
size_t lio_send_bt(char* message, uint32_t size)
{
    for(int i = 0; i < size && i < LIO_BT_BUFFER_SIZE; i++) {
        tx_dma_buffer[i] = message[i];
    }

    dma_parameter_struct dma_init_struct;
    dma_deinit(DMA0, DMA_CH3);
    dma_init_struct.direction = DMA_MEMORY_TO_PERIPHERAL;
    dma_init_struct.memory_addr = (uint32_t)tx_dma_buffer;
    dma_init_struct.memory_inc = DMA_MEMORY_INCREASE_ENABLE;
    dma_init_struct.memory_width = DMA_MEMORY_WIDTH_8BIT;
    dma_init_struct.number = 1;
    dma_init_struct.periph_addr = (uint32_t)&USART_DATA(USART0);
    dma_init_struct.periph_inc = DMA_PERIPH_INCREASE_DISABLE;
    dma_init_struct.periph_width = DMA_PERIPHERAL_WIDTH_8BIT;
    dma_init_struct.priority = DMA_PRIORITY_ULTRA_HIGH;
    dma_init(DMA0, DMA_CH3, &dma_init_struct);
    /* configure DMA mode */
    dma_circulation_disable(DMA0, DMA_CH3);
    /* enable DMA channel3 */
    dma_channel_enable(DMA0, DMA_CH3);
}
```

Enable

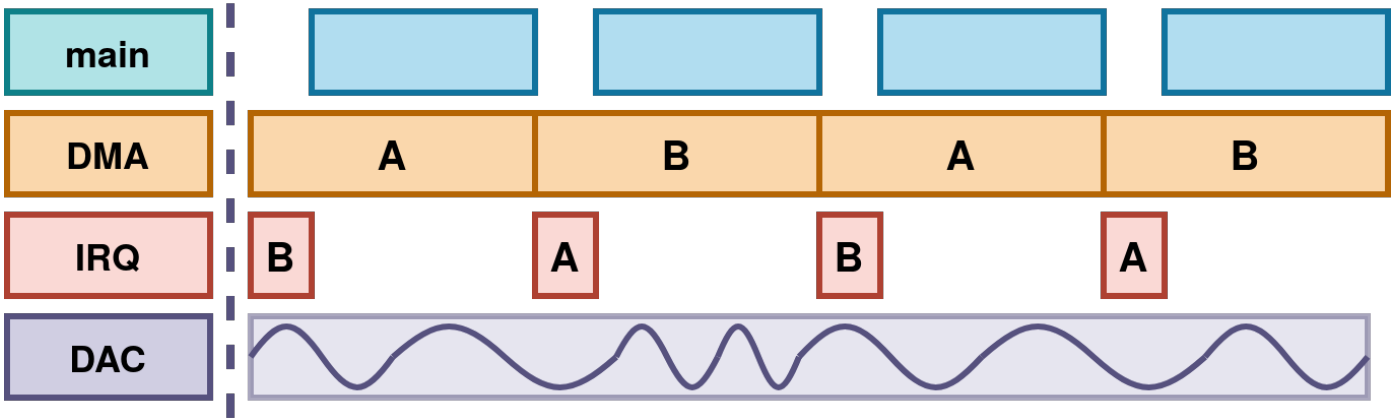
Message will go out automatically without any more code!

SPI – Callback interrupt



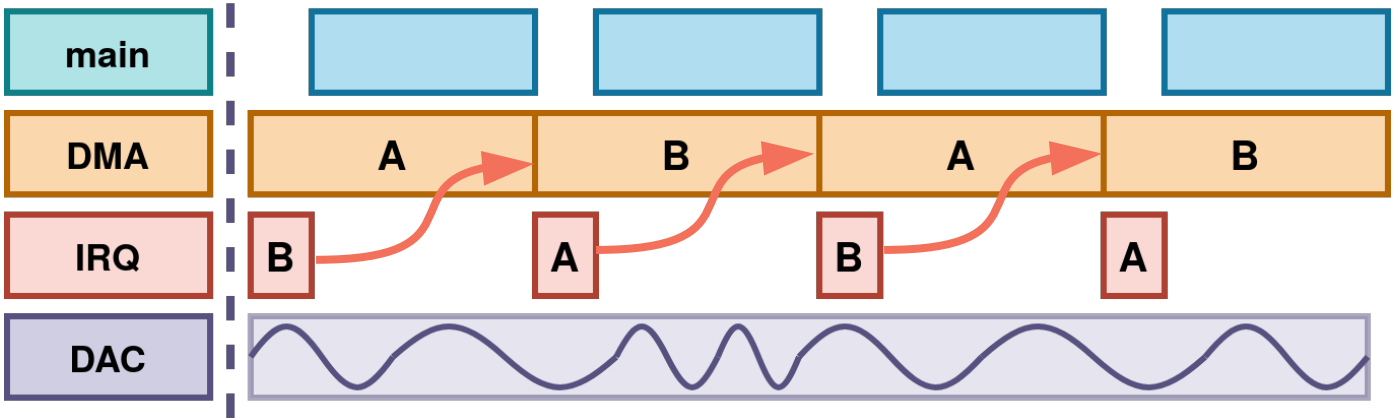
DAC – Double buffer

* AKA Ping Pong buffer,
or Page turn buffer



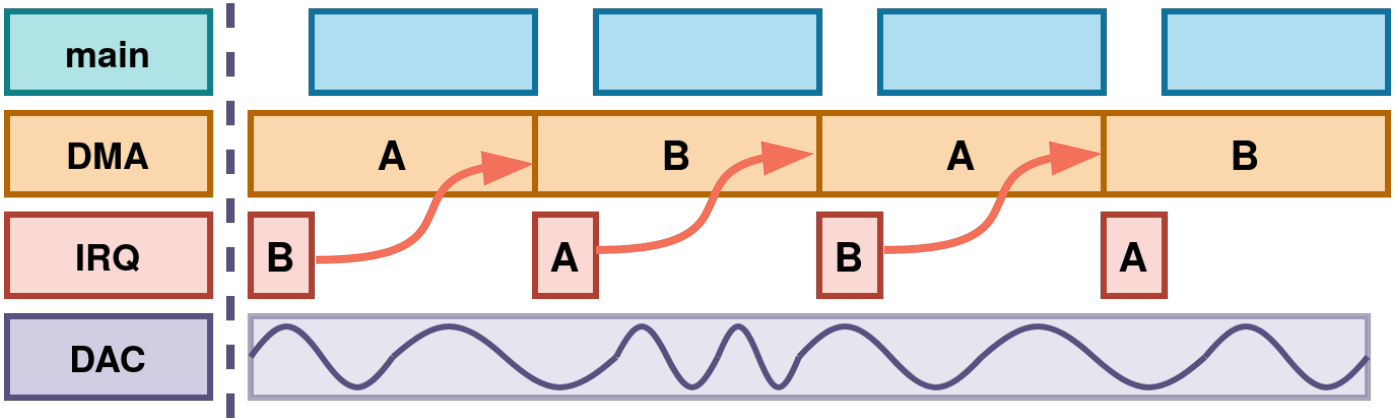
DAC – Double buffer

* AKA Ping Pong buffer,
or page flipping



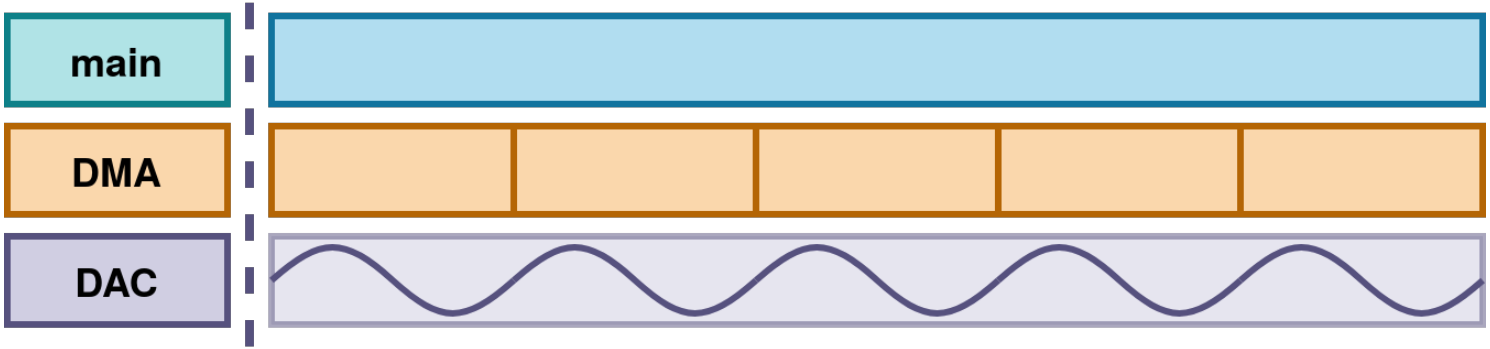
DAC – Double buffer

* AKA Ping Pong buffer,
or Page turn buffer



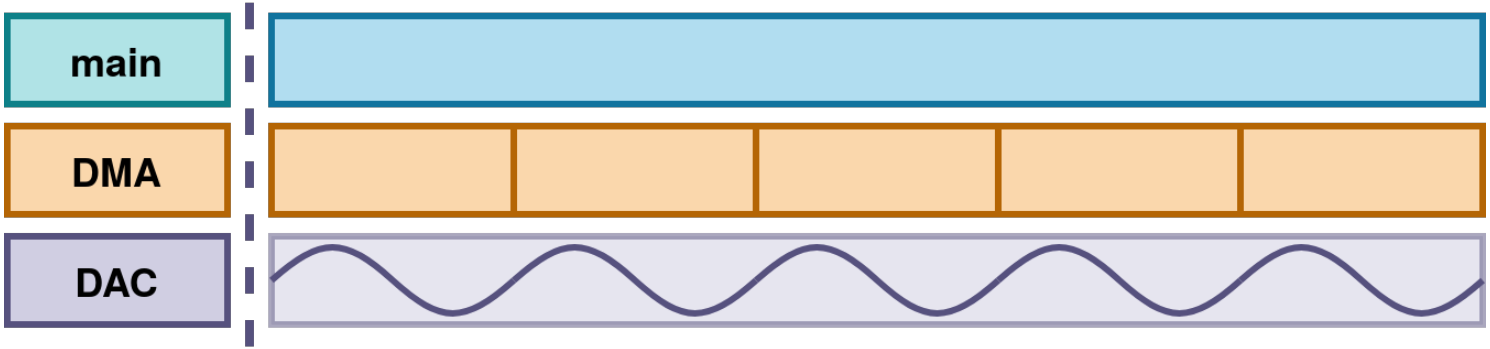
Introduces some latency and memory in exchange for consistency between samples.

DAC – Continuous



DAC – Continuous

“Set and forget”



Essentially zero processing overhead!

DMA - summary

For:

DMA - summary

For:

High bandwidth

DMA - summary

For:

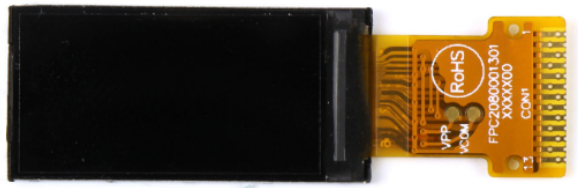
High bandwidth

Sporadic events

DMA - summary

For:

High bandwidth



Sporadic events

Timing sensitive